QuantPulse 数据流学习速记

01 / 后端与数据保存 根据当前项目整理 · 2026-10-29

1 语言、包和函数分别是什么

后端 .py 文件使用 **Python**；App.jsx 使用 **JavaScript / JSX**；CSS 控制颜色和布局。yfinance、FastAPI 是安装的 Python 包，React 是前端库。

```
import yfinance as yf
ticker = yf.Ticker("AAPL")
history = ticker.history(...)
```

import 让当前文件能使用这个包；as yf 给它取简称。Yahoo 的地址和请求方式写在 yfinance 内部，调用 history(...) 才查询行情。**try / except** 用来尝试执行并处理错误，本身不负责联网。

2 内存、变量和文件不要混在一起

内存（RAM）是程序运行时临时存放内容的地方。写 price = 100 不会生成 price 文件；history 也是变量，只是对应的内容是一张 pandas DataFrame（表格）。

ticker	内存中的股票查询对象，不是下载文件。
history / prices / saved_history	内存表格：下载结果 / 清洗结果 / 从 CSV 重新读取的结果。
csv_path / prices.csv	前者是文件位置；后者是磁盘文件，程序退出后仍保留。

return history, currency 把表格和币种（如 USD）交回调用方；history, currency = download_history(...) 用两个变量接住它们。函数返回后，调用方继续执行后面的代码。

3 FastAPI 如何处理一次历史查询

启动后端时，**@app.get("/history")** 建立“地址对应函数”的规则。请求到达后，FastAPI 把代码、日期交给 main.py 中的 get_history()；Uvicorn 负责让应用在端口上持续运行。

get_history() 先检查参数，再调用 **market_data.py / download_history()** 下载表格。随后调用 **snapshots.py / save_snapshot()**：其中 prepare_history() 只负责排序、去重和处理缺失值，save_snapshot() 负责写文件。下载函数本身不保存 CSV。

```
saved_history = pd.read_csv(csv_path)
result["data"] = saved_history.to_dict(orient="records")
return result
```

上面三步在 main.py 第 104、112、122 行：**读 CSV 成表格 → 每行转字典，组成列表并放进结果字典 → 交回 FastAPI。**FastAPI 内部自动把 Python 字典转换成 JSON 响应发回浏览器；这里不需要生成 JSON 文件。

4 快照会保存多久，为什么文件增多

当前每次成功查询有记录时，都会在 backend/data/ 下新建带获取时间的目录，保存 prices.csv 与 metadata.json。旧快照不会覆盖，也不会自动更新；它可以长期使用。**目前没有自动清理，也没有自动复用旧快照的行情缓存。**

后续可将“日常查询”和“需要时保存快照”分开，或设置缓存与保留期限；这些是控制文件增长的建议，尚未实现。

前端如何把结果显示成表格

02 / 从输入框到 React state，再到页面中的每一行

5 Search 和 parameters 是怎么连起来的

Search 按钮提交表单，onSubmit={searchStock} 调用前端函数。event.preventDefault() 阻止表单默认刷新。输入框 state 保存 symbol、startDate、endDate。

```
const parameters = new URLSearchParams({
  symbol: symbol, start: startDate, end: endDate,
})
```

parameters 是我们起的变量名。URLSearchParams 把参数整理成网址查询部分；左侧 symbol / start / end 是后端要接收的名字，右侧是前端变量的值。最终 fetch 请求类似：

```
GET /history?symbol=AAPL&start=2024-01-02&end=2024-01-09
```

API 的开始日和结束日都包含；后端给结束日加一天，再交给使用“不包含结束日”规则的 Yahoo。周末、休市日可能没有记录，不补造价格。

6 body、result 与 setResult 的关系

后端 result 是 **Python 字典**；网络传来的 JSON 经 response.json() 解析后，成为 **JavaScript 对象 body**。它包含股票说明和 data 数组，不只是一堆没有结构的数字。

```
const [result, setResult] = useState(null)
// After a successful response:
setResult(body)
```

useState 提供“当前值”和“更新函数”；**setResult 不是另写的自定义函数**，而是我们给 React 返回的更新函数起的名字。调用它会记住整份 body，并安排重新渲染 App。下次渲染的 result 就能读取这份对象；useState(null) 不会因重复渲染而重置结果。

7 问号、冒号、rows 和 row 是什么意思

```
const rows = result ? result.data : []
```

条件表达式是“条件 ? 成立时的值 : 不成立时的值”。这里表示：有 result 就取逐日记录 result.data；还没有结果就用空数组 []。**rows 是多条记录的数组；row 是 map 每次拿到的其中一条记录。**

```
rows.slice(0, 10).map((row) => (
  <tr key={row.date}>
    <th scope="row">{row.date}</th>
    <td>{row.close}</td>
  </tr>
))
```

上面仅保留两列示意。slice(0, 10) 取前 10 条；map 为每条记录生成一个 tr（表格行），th / td 是单元格，JSX 的 {row.close} 把该记录的值放进页面。完整记录仍在 result.data；显示两位小数不会修改保存的数据。

8 复习时打开哪些文件

App.jsx：输入与请求；第 59 行更新 state，第 73 行取记录，第 205 行生成表格行。

App.css 管三栏及控件，**index.css** 管全局字体和颜色；className="workspace" 对应 CSS 的 .workspace。**index.html** 管浏览器标签标题与页面简介。

一条完整路径：Search → /history → 下载与清洗 → CSV → JSON 响应 → body → setResult → rows.map → 页面表格。